

1320. Minimum Distance to Type a Word Using Two Fingers

Solved 

Hard

 Topics

 Companies

 Hint

A	B	C	D	E	F
G	H	I	J	K	L
M	N	O	P	Q	R
S	T	U	V	W	X
Y	Z				

You have a keyboard layout as shown above in the **X-Y** plane, where each English uppercase letter is located at some coordinate.

- For example, the letter 'A' is located at coordinate $(0, 0)$, the letter 'B' is located at coordinate $(0, 1)$, the letter 'P' is located at coordinate $(2, 3)$ and the letter 'Z' is located at coordinate $(4, 1)$.

Given the string `word`, return *the minimum total **distance** to type such string using only two fingers*.

The **distance** between coordinates (x_1, y_1) and (x_2, y_2) is $|x_1 - x_2| + |y_1 - y_2|$.

Note that the initial positions of your two fingers are considered free so do not count towards your total distance, also your two fingers do not have to start at the first letter or the first two letters.

Example 1:

Input: word = "CAKE"

Output: 3

Explanation: Using two fingers, one optimal way to type "CAKE" is:

Finger 1 on letter 'C' -> cost = 0

Finger 1 on letter 'A' -> cost = Distance from letter 'C' to letter 'A' = 2

Finger 2 on letter 'K' -> cost = 0

Finger 2 on letter 'E' -> cost = Distance from letter 'K' to letter 'E' = 1

Total distance = 3

Example 2:

Input: word = "HAPPY"

Output: 6

Explanation: Using two fingers, one optimal way to type "HAPPY" is:

Finger 1 on letter 'H' -> cost = 0

Finger 1 on letter 'A' -> cost = Distance from letter 'H' to letter 'A' = 2

Finger 2 on letter 'P' -> cost = 0

Finger 2 on letter 'P' -> cost = Distance from letter 'P' to letter 'P' = 0

Finger 1 on letter 'Y' -> cost = Distance from letter 'A' to letter 'Y' = 4

Total distance = 6

Constraints:

- `2 <= word.length <= 300`
- `word` consists of uppercase English letters.

Python:

class Solution:

def minimumDistance(self, word: str) -> int:

def dist(a,b):

if a == 26 or b == 26:

return 0

return abs(a//6-b//6) + abs(a%6-b%6)

n = len(word)

A = ord('A')

idx = [ord(c)-A for c in word]

dp = [[float('inf')]*27 for _ in range(27)]

dp[26][26] = 0

for c in idx:

ndp = [[float('inf')]*27 for _ in range(27)]

for i in range(27):

for j in range(27):

```

        v = dp[i][j]
        if v >= float('inf'):
            continue
        if v+dist(i,c) < ndp[c][j]:
            ndp[c][j] = v + dist(i,c)
        if v+dist(j,c) < ndp[i][c]:
            ndp[i][c] = v + dist(j,c)
    dp = ndp
    return min(min(row) for row in dp)

```

JavaScript:

```

const minimumDistance = (t) => {
    const n = t.length;

    const dp = new Array(n - 1);    // DP table
    const cost = new Uint16Array(n); // cumulative cost

    // Initialize DP rows and base cost
    for (let i = 1; i < n; ++i) {
        dp[i - 1] = new Uint16Array(n);
        cost[i] += cost[i - 1] + distance(i - 1, i);
    }

    // Fill DP table
    for (let i = 2; i < n; ++i) {
        let minVal = cost[i - 1];

        for (let j = 0; j < i - 1; ++j) {
            const candidate = dp[j][i - 1] + distance(j, i);

            if (candidate < minVal) {
                minVal = candidate;
            }

            dp[j][i] = dp[j][i - 1] + distance(i - 1, i);
        }

        dp[i - 1][i] = minVal;
    }

    // Find final result
    let result = cost[n - 1];

    for (let i = 0; i < n - 1; ++i) {

```

```

    if (dp[i][n - 1] < result) {
        result = dp[i][n - 1];
    }
}

return result;

// Helper function to calculate distance
function distance(i, j) {
    const x1 = t.charCodeAt(i) - 65;
    const x2 = t.charCodeAt(j) - 65;

    return (
        Math.abs(x1 % 6 - x2 % 6) +
        Math.abs(((x1 / 6) << 0) - ((x2 / 6) << 0))
    );
}
};

```

Java:

```

class Solution {

    // distance between two chars
    private int dist(char a, char b) {
        int x1 = (a - 'A') / 6, y1 = (a - 'A') % 6;
        int x2 = (b - 'A') / 6, y2 = (b - 'A') % 6;
        return Math.abs(x1 - x2) + Math.abs(y1 - y2);
    }

    public int minimumDistance(String word) {
        int n = word.length();

        // dp[j] = max saving when second finger is at j
        int[] dp = new int[26];
        int total = 0;

        for (int i = 0; i < n - 1; i++) {
            char a = word.charAt(i);
            char b = word.charAt(i + 1);

            int d = dist(a, b);
            total += d;

            int[] newDp = new int[26];

```

```

    for (int j = 0; j < 26; j++) {
        // case1: don't use second finger
        newDp[j] = Math.max(newDp[j], dp[j]);

        // case2: use second finger
        int saving = dp[j] + d - dist((char)(j + 'A'), b);
        newDp[a - 'A'] = Math.max(newDp[a - 'A'], saving);
    }

    dp = newDp;
}

int maxSaving = 0;
for (int val : dp) {
    maxSaving = Math.max(maxSaving, val);
}

return total - maxSaving;
}
}

```